12. Numpy

- **numpy 설치**

> pip install numpy
> pip install numpy==1.23.5

- **numpy 설치 확인**

```
import numpy
print(numpy.__version__)

import numpy as np
print(np.__version__)
```

- array
  - numpy의 array는 동일한 타입으로만 이루어진다. 이는, numpy의 연산을 매우 빠르게 진행하기 위한 것이다
  - 배열은 NumPy 라이브러리의 중앙 데이터 구조라고 할 수 있다. 배열은 values 및 원시 데이터에 대한 정보, 요소를 찾는 방법, 요소를 해석하는 방법. 인덱싱할 수 있는 요소 그리드가 있다.

```
//  array 만들기
import numpy as np
range_array = np.arange(10)
print(range_array)
>---
[0 1 2 3 4 5 6 7 8 9]

import numpy as np
a = np.array([1, 2, 3, 4, 5])
print(a)

[1 2 3 4 5]

// NumPy 배열의 크기를 확인
```

```
print(a.shape)
print(range_array.shape)

// NumPy 배열 연산
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# 원소별 덧셈
c = a + b  # [5, 7, 9]

# 원소별 곱셈
d = a * b  # [4, 10, 18]

# 스칼라와의 연산
e = a + 1  # [2, 3, 4]
```

```
// NumPy 배열 형태 변경
a = np.array([[1, 2], [3, 4], [5, 6]])

# 배열의 형태 변경
b = a.reshape((2, 3))  # [[1, 2, 3], [4, 5, 6]]

# 배열 전치
c = a.T  # [[1, 3, 5], [2, 4, 6]]

// NumPy 배열 병합과 분리
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# 배열 병합
c = np.concatenate((a, b))  # [1, 2, 3, 4, 5, 6]

// 다차원 NumPy 배열을 병합
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6]])

# 배열 병합
```

```
c = np.concatenate((a, b), axis=0)  # [[1, 2], [3, 4],
[5, 6]]

// 배열을 분리
a = np.array([[1, 2, 3], [4, 5, 6]])

# 배열 분리
b, c = np.split(a, [1], axis=0)  # [[1, 2, 3]], [[4, 5,
6]]
```

- **NumPy 주요 함수**
  - `np.zeros()`: 모든 원소가 0인 배열을 생성합니다.
  - `np.ones()`: 모든 원소가 1인 배열을 생성합니다.
  - `np.arange()`: 범위 내의 일정 간격을 가진 배열을 생성합니다.
  - `np.linspace()`: 범위 내에서 균등 간격으로 원하는 개수의 배열을 생성합니다.
  - `np.random.random()`: 0부터 1사이의 난수를 가지는 배열을 생성합니다.
  - `np.random.randn()`: 평균이 0이고 표준편차가 1인 정규 분포를 따르는 난수를 가지는 배열을 생성합니다.

```
import numpy as np

arr = np.zeros((2, 3))
print(arr)
---
 [[0. 0. 0.]
 [0. 0. 0.]]

 //
 np.ones((3, 5))

array([[1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.]])

//
arr = np.arange(1, 10, 2)
```

```
print(arr)
---
[1 3 5 7 9]
//
arr = np.linspace(0, 1, 5)
print(arr)
---
[0.   0.25 0.5  0.75 1.  ]

//
arr = np.random.random((3, 3))
print(arr)
---
[[0.8139928  0.79544491 0.43146849]
 [0.30136094 0.77515306 0.64259144]
 [0.02956163 0.74888294 0.56169272]]

//
 arr = np.random.randn(2, 4)
 print(arr)
 ---
 [[ 0.52346546  0.30785207  0.63301317 -0.03108197]
 [ 1.37352221 -1.26632616 -1.15567608 -0.52687134]]
```

- **Numpy 수학 함수**
  - `sum()`, `mean()` : 배열 전체 합, 평균
  - `cumsum()`, `cumprod()` : 배열 누적 합, 누적 곱
  - `std()`, `var()` : 표준편차, 분산
  - `min()`, `max()` : 최소값, 최대값
  - `argmin()`, `argmax()`: 최소 원소의 색인 값, 최대 원소의 색인 값
- **Numpy 난수 함수**
  - `seed()` : 난수 발생기의 seed를 지정한다.
  - `permutation()` : 임의의 순열을 반환한다.
  - `shuffle()` : 리스트나 배열의 순서를 뒤섞는다.

- `rand()` : 균등분포에서 표본을 추출한다.
- `randint()` : 주어진 최소/최대 범위 안에서 임의의 난수를 추출한다.
- `randn()` : 표준편차가 1이고 평균값이 0인 정규분포에서 표본을 추출한다.
- `binomial()` : 이항분포에서 표본을 추출한다.
- `normal()` : 정규분포(가우시안)에서 표본을 추출한다.
- `beta()` : 베타분포에서 표본을 추출한다.
- `chisquare()` : 카이제곱분포에서 표본을 추출한다.
- `gamma()` : 감마분포에서 표본을 추출한다.
- `uniform()` : 균등(0,1)에서 표본을 추출한다.